

# Working with CSV Files

Module 15 Study Notes • Data Ingestion with Pandas

## 1. Introduction to CSV Ingestion

---

When starting an ML project, 90% of the datasets you encounter will be in a Comma-Separated Values (CSV) format. The Pandas library provides a highly powerful function, `pd.read_csv()`, to load this data into a DataFrame. However, real-world data is messy, requiring advanced parameter tuning to load correctly.

## 2. Loading Data (Local vs. Remote)

---

- **Local Files:** `pd.read_csv('filename.csv')`
- **Remote URLs:** You can pass a direct URL to a CSV file. If the server requires headers, you can use the `requests` library to fetch the raw text and then parse it using `io.StringIO`.

## 3. Essential `read\_csv` Parameters

---

To handle messy datasets, you must master the various parameters of `read_csv()`:

### A. Handling Structure & Delimiters

- **sep**: By default, Pandas splits data by commas. If working with TSV (Tab-Separated Values), use `sep='\t'`.
- **names**: If a CSV lacks a header row, Pandas will accidentally turn the first row of data into column names. Pass a list of strings to `names` to define custom column headers.
- **header**: If the column names are accidentally placed on a lower row (e.g., row 1 instead of row 0), use `header=1` to skip the junk data above it.
- **index\_col**: Prevents Pandas from generating a default index (0, 1, 2...) if your data already has a primary key column (e.g., `index_col='id'`).

### B. Filtering & Memory Optimization

- **usecols**: If a dataset has 100 columns but you only need 3, pass `usecols=['col1', 'col2', 'col3']`. This drastically reduces memory consumption.

- **nrows** : Loads only the first  $N$  rows. Great for quickly inspecting massive datasets without crashing your computer.
- **skiprows** : Ignores specific rows (e.g., **skiprows=[0, 2, 5]** ) or can accept a function to filter rows dynamically.

## C. Handling Data Integrity & Formatting

- **encoding** : If a dataset contains special characters (like emojis or non-English text) and throws a *UnicodeDecodeError*, change the encoding (e.g., **encoding='latin-1'** ).
- **error\_bad\_lines** / **on\_bad\_lines** : If a few rows in the CSV are corrupted (e.g., have 6 values instead of 5), Pandas will crash. Setting this to False/Skip ignores the corrupted rows and loads the rest safely.
- **dtype** : Forces Pandas to load a column as a specific data type (e.g., converting a float to an integer to save memory).
- **parse\_dates** : By default, dates are loaded as Strings (Objects). Passing **parse\_dates=['date\_column']** converts them into actual Datetime objects, enabling time-series operations.
- **na\_values** : Specifies custom strings that should be treated as NaN (missing data), such as **na\_values=['-', 'Missing']** .
- **converters** : Allows you to apply a Python function to a column \*during\* the load process to clean or transform the data immediately.

### Handling Massive Datasets (Out-of-Core Processing):

If a dataset is larger than your computer's RAM (e.g., 10GB data on an 8GB RAM machine), use the **chunksize** parameter.

Example: **pd.read\_csv('massive.csv', chunksize=5000)** .

This returns an iterator, allowing you to load and process the data in small, manageable blocks of 5,000 rows at a time inside a for-loop.